

PRIMARY KEY — первичный ключ

Зачем он нужен: аналогия из жизни

Представьте студенческую группу, где есть два студента с именем "Иван Иванов". Если преподаватель скажет "поставь зачёт Ивану Иванову" — непонятно, кому именно. Но у каждого студента есть **номер студенческого билета** — уникальный, не меняется, есть у всех. Именно так работает первичный ключ в таблице: это столбец (или несколько), который **однозначно и навсегда** определяет строку, даже если остальные данные (имя, телефон, адрес) совпадают у нескольких записей или меняются со временем.

Три требования к первичному ключу

- 1. Уникальность** — среди всех строк таблицы не может быть двух с одинаковым значением ключа.
- 2. Обязательность** — значение должно быть у каждой строки, **NULL** недопустим.
- 3. Неизменность** — в идеале значение не меняется никогда после создания строки (в отличие от имени, телефона или адреса, которые вполне могут поменяться).

Именно поэтому в качестве первичного ключа почти никогда не берут "естественные" данные вроде ФИО, email или номера телефона — они не гарантируют уникальность (тёзки) и могут измениться (смена телефона, фамилии). Вместо этого создают отдельный столбец-идентификатор, который не несёт никакого смысла, кроме "это строка номер такая-то" — обычно **id**.

Синтаксис в SQLite

```
CREATE TABLE students (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  full_name TEXT NOT NULL,  
  phone TEXT  
);
```

При вставке строки **id** можно не указывать — SQLite подставит следующее свободное целое число сам:

```
INSERT INTO students (full_name, phone) VALUES ('Иван Иванов',  
' +998901234567');  
INSERT INTO students (full_name, phone) VALUES ('Иван Иванов',  
' +998907654321'); -- тёзка, но id разный  
SELECT * FROM students;  
-- 1 | Иван Иванов | +998901234567  
-- 2 | Иван Иванов | +998907654321
```

Несмотря на полное совпадение имени, это два разных, однозначно различимых студента — благодаря **id**.

Попытка нарушить уникальность

```
INSERT INTO students (id, full_name) VALUES (1, 'Другой человек');  
-- Error: UNIQUE constraint failed: students.id
```

SQLite не позволит вставить вторую строку с уже занятым **id** — это и есть работа первичного ключа на практике.

AUTOINCREMENT — гарантия, что id никогда не

повторится

Если объявить столбец просто как `id INTEGER PRIMARY KEY` (без `AUTOINCREMENT`), SQLite всё равно подставляет следующее число сам, но может **повторно использовать id удалённой строки** — например, если строка с `id=5` была самой большой и её удалили, следующая вставленная строка снова может получить `id=5`.

`AUTOINCREMENT` — необязательное дополнение, которое меняет это поведение: `id` **всегда только растёт** и никогда не переиспользуется, даже после удаления строк.

```
CREATE TABLE students (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    full_name TEXT NOT NULL  
);
```

Разница важна, когда на `id` где-то ссылаются извне — например, номер студенческого в бумажном журнале, ссылка в чате, ключ в другой таблице (это и есть `FOREIGN KEY`, следующая тема). Если `id` переиспользуется, старая ссылка может внезапно "перепутаться" и указать на новую, не имеющую отношения строку. Расплата — таблица чуть медленнее на вставках, поэтому `AUTOINCREMENT` стоит добавлять только тогда, когда эта гарантия действительно нужна, а не по умолчанию везде.

Важно: `AUTOINCREMENT` работает только с `INTEGER PRIMARY KEY` на одном столбце. У составного ключа (пример ниже) или у "естественного" ключа-строки (`id TEXT PRIMARY KEY`, как студенческий билет `st-web-869`) автоинкремента нет и быть не может — там `id` и так не является просто "порядковым номером".

"Естественный" ключ vs суррогатный ключ

Иногда в реальном мире уже есть уникальный идентификатор — например, номер паспорта, ИНН, номер студенческого билета (`st-web-869`, как в этой самой платформе). Такой ключ называют **естественным**, и его можно использовать как первичный напрямую, без отдельного `id`:

```
CREATE TABLE users (  
    id TEXT PRIMARY KEY, -- здесь id и есть тот самый студенческий, например  
    'st-web-869'  
    name TEXT NOT NULL  
);
```

Если же подходящего природного уникального признака нет (или он теоретически может измениться) — создают **суррогатный** ключ: отдельный технический столбец `id INTEGER PRIMARY KEY AUTOINCREMENT`, который ничего не значит сам по себе, кроме "номер строки".

Составной первичный ключ

Иногда уникальность строки обеспечивается только сочетанием нескольких столбцов. Пример: таблица "оценка студента за задание" — один и тот же студент может иметь много оценок, одно и то же задание — много оценок у разных студентов, но пара (студент, задание) должна быть уникальной:

```
CREATE TABLE grades (  
    student_id INTEGER NOT NULL,  
    assignment_id INTEGER NOT NULL,  
    score INTEGER NOT NULL,  
    PRIMARY KEY (student_id, assignment_id)  
);
```

Частые ошибки

- Выбор в качестве первичного ключа данных, которые могут повториться (имя) или измениться (телефон, email) — со временем это почти всегда создаёт проблемы.
- Ожидание, что SQLite сам "заметит" логическую уникальность (например, одинаковый email) — без явного **PRIMARY KEY** или **UNIQUE** это не проверяется автоматически.
- Добавление **AUTOINCREMENT** туда, где он не нужен (просто "на всякий случай") — лишние затраты на вставку без реальной необходимости.
- Путаница между первичным ключом (внутри одной таблицы) и внешним ключом (связь с другой таблицей) — следующая тема разбирает это подробно.